Towards Pulverised Architectures for Collective Adaptive Systems through Multi-tier Programming

Gianluca Aguzzi[1] Roberto Casadei[1] Danilo Pianini[1]
Guido Salvaneschi[2] Mirko Viroli[1]

[1]Alma Mater Studiorum – Università di Bologna, Cesena, Italy
[2]University of St.Gallen: St.Gallen, Switzerland

October 1, 2021

Talk @ **eCAS 2021**
6th Workshop on Engineering Collective Adaptive Systems



Collective Adapative Systems: Challenges

- Complex and layered networks
  - Large scale
  - Heterogenous
  - Dynamic
- Distributed control
- Environmental changes

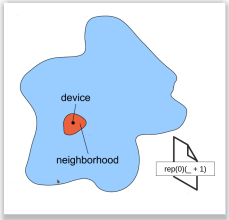
Engineering problem

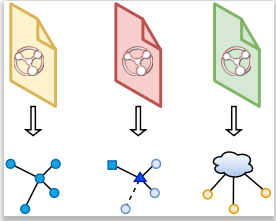
- Is it possible to define a collective behaviour indipedently from the network topology?

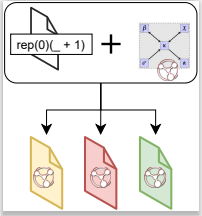
Proposed approach

- Describe the collective behaviour with *Aggregate Computing*[a]
- Inject the *same* collective behaviour in different deployments thanks to *Pulverisation*[b]
- Define your concrete deployment with *Multi-tier programming*[c]

[a] Mirko Viroli et al. "From distributed coordination to field calculus and aggregate computing". (2019).
[b] Roberto Casadei et al. "Pulverization in Cyber-Physical Systems: Engineering the Self-Organizing Logic Separated from Deployment". (2020).
[c] Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. "A Survey of Multitier Programming". (2020).

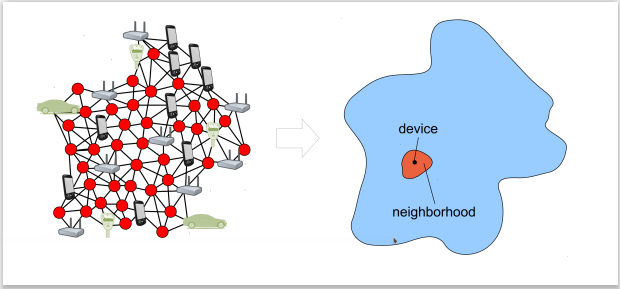


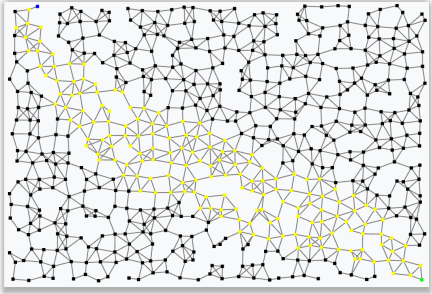




Aggregate Computing

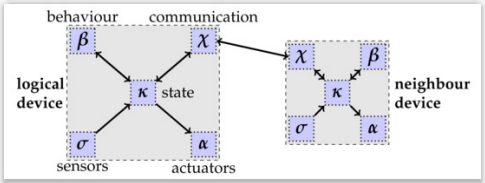
- Top-down collective self-organising behaviour specification
- Applied to possible very large scale system
- Interaction protocols implicitly generated
- Peer-to-peer network structure assumption

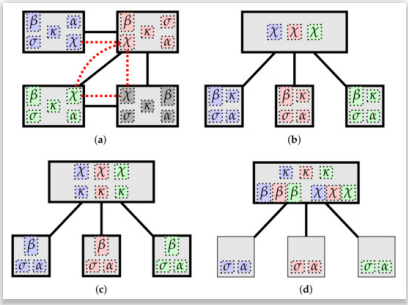




Pulverisation

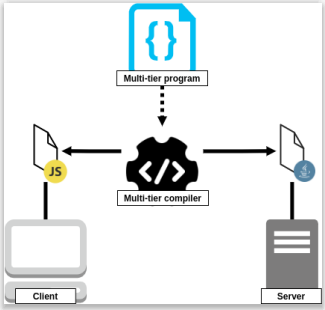
- Separation between behavioural and deployments aspects
- Logical devices decomposed in micro-components
- Interaction protocols defined at deployment time
- Does not provide how to specify and deploy component is a safe and meaningful way





Multi-tier programming

- Distributed architectures declaratively described in a single code base
- The correctness and coherence of the architecture is veryfied by the compiler in the type-level language
- Behavioural (functional) and structural (non-functional) aspect are mixed togheter



Frameworks

ScaFi (Scala Field)

A tool-chain comprised by a `DSL` that supports field-calculus. [1]

ScalaLoci

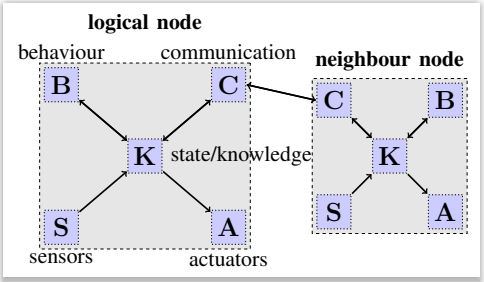
A Scala langauge extension to enable compile-time multi-tier programming. [2]

[1] Roberto Casadei et al. “FScaFi : A Core Calculus for Collective Adaptive Systems Programming”. 2020

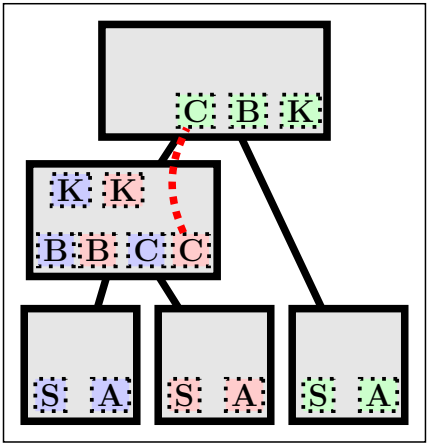
[2] Pascal Weisenburger, Mirko Köhler, and Guido Salvaneschi. “Distributed system development with ScalaLoci”. (2018)

Puliverised architectures: **logical components**

```
@multitier trait LogicalSystem {
  @peer type LNode <: { type Tie <: Multiple[LNode] }
}
@multitier trait PulverisedSystem
 extends LogicalSystem {
  @peer type SensorComponent         <: LNode
  @peer type ActuatorComponent       <: LNode
  @peer type StateComponent          <: LNode
  @peer type BehaviourComponent      <: LNode
  @peer type CommunicationComponent <: LNode
}
```



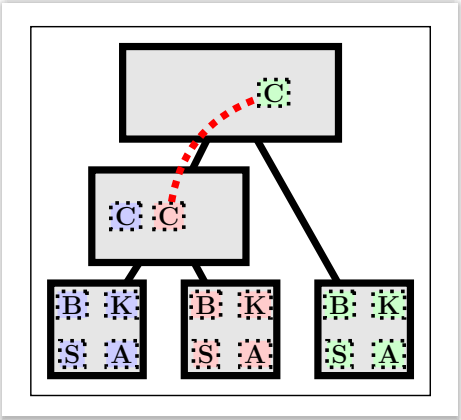
Puliverised architectures: **concrete deployments**




```
@multitier object IoTSystem extends PulverisedSystem {
  @peer type Thin <: { type Tie <: Multiple[Thick] }
  @peer type Thick <: {
    type Tie <: Multiple[Thick] with Multiple[Thin]
  }
  // Pulverised component allocation on devices
  @peer type SensorComponent <: Thin
  @peer type ActuatorComponent <: Thin
  @peer type StateComponent <: Thick
  @peer type BehaviourComponent <: Thick
  @peer type CommunicationComponent <: Thick
}
```

Puliverised architectures: **concrete deployments**

```
@multitier object BrokerBased extends PulverisedSystem {
  @peer type Node <: { type Tie <: Single[Broker] }
  @peer type Broker <: {
    type Tie <: Multiple[Node] with Multiple[Broker]
  }
  // Pulverised component allocation on devices
  @peer type SensorComponent <: Node
  @peer type ActuatorComponent <: Node
  @peer type StateComponent <: Node
  @peer type BehaviourComponent <: Node
  @peer type CommunicationComponent <: Broker
}
```



Conclusion

Benefits

- Collective behaviours are defined regardless of the IT architecture
- Deployments specification is declarative and type-safe thanks to Scala Loci

Future Works

- Opportunistic deployment and reconfiguration of the pulverised system
- Bring this example from a prototype to a usable library

References

[1] Mirko Viroli et al. “From distributed coordination to field calculus and aggregate computing”. *J. Log. Algebraic Methods Program.* 109 (2019). DOI: 10.1016/j.jlamp.2019.100486.

[2] Roberto Casadei et al. “Pulverization in Cyber-Physical Systems: Engineering the Self-Organizing Logic Separated from Deployment”. *Future Internet* 12.11 (2020), p. 203. DOI: 10.3390/fi12110203.

[3] Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. “A Survey of Multitier Programming”. *ACM Comput. Surv.* 53.4 (2020), 81:1–81:35. DOI: 10.1145/3397495.

[4] Roberto Casadei et al. “FScaFi : A Core Calculus for Collective Adaptive Systems Programming”. *ISoLA (2)*. Vol. 12477. LNCS. Springer, 2020, pp. 344–360.

[5] Pascal Weisenburger, Mirko Köhler, and Guido Salvaneschi. “Distributed system development with ScalaLoci”. *Proc. ACM Program. Lang.* 2.OOPSLA (2018), 129:1–129:30. DOI: 10.1145/3276499.